

软件入门指南

EIC7x 系列 AI 数字 SoC

Engineering Draft / Rev 1.11
2025-01-25

声明

版权所有©2024，北京奕斯伟计算技术股份有限公司（以下简称“ESWIN”）。保留所有权利。

ESWIN 在该产品中保留所有知识产权和专有权利。未经 ESWIN 授权，不得发布、复制、分发、转载、修改、改编、翻译或制作此软件的衍生作品，无论是全部还是部分。

对于本文档的任何修改，ESWIN 无需承担通知义务且不代表 ESWIN 做出任何承诺。除非另有说明，本文档中的所有陈述、信息及建议均按“现状”提供，ESWIN 不对此做出任何形式的担保、保证与承诺，无论明示或默示。（本段落用于文档中）

“ESWIN” 商标、logo 和其他 ESWIN 标识，为北京奕斯伟计算技术股份有限公司及（或）其母公司所有的商标。

本文档提及的其他所有商标或商品名称，由其各自的所有权人所有。

修订记录

文档版本	日期	责任人	说明
V1.11	2025/01/25	毛伟、罗伟洋	更新 20251230 交付版本信息
V1.10	2025/10/29	毛伟、罗伟洋	更新 20251030 交付版本信息
V1.9	2025/09/05	毛伟、吕金刚	更新 20250730 交付版本信息
V1.8	2025/7/18	毛伟、吕金刚	更新 20250630 交付版本信息
V1.7	2025/6/12	毛伟、吕金刚	更新 20250330、20250530 交付版本信息
V1.6	2025/3/13	毛伟、吕金刚	更新 20250228 交付版本信息
V1.5	2025/2/17	毛伟、吕金刚	更新 20250130 交付版本信息
V1.4	2024/12/31	毛伟、吕金刚、王渊	修改开发板型号信息、Deb 包信息
V1.3	2024/10/22	毛伟、吕金刚、杨洋	增加第四章配置远端开发环境内容
V1.2	2024/10/22	毛伟、吕金刚、杨洋	增加 ubuntu20.04 系统上使用 docker 编译
v1.1	2024/8/10	毛伟、吕金刚、杨洋	更新了发布的 deb 包清单、更新了系统编译的示例图。
v1.0	2024/8/10	毛伟、吕金刚、杨洋	建立初稿。

目 录

软件入门指南.....	0
EIC7x 系列 AI 数字 SoC	0
1. 介绍 INTRODUCTION	1
2. EIC7X 软件栈完整生态整体介绍.....	2
2.1 DEBIAN 发行版的组织形式.....	2
2.2 EIC7x 软件栈为用户提供的专属工具.....	3
2.2.1 Eswin 提供的专属 deb 包.....	3
2.2.2 X86 平台上使用的工具清单	5
2.2.3 为用户单独提供的 Docker 镜像	6
3. EIC7X 软件栈使用.....	6
3.1 环境搭建.....	6
3.2 烧写	7
3.3 系统编译.....	7
3.4 HELLO WORLD 示例程序	9
3.4.1 交叉编译	9
3.4.2 构建 deb 包	10
3.4.3 交叉编译工具链扩展	11
3.5 EIC7x 软件栈中 ESWIN 提供的专属 DEB 包的使用方法及注意事项	13
3.5.1 安装软件 Deb 包	13
3.6 定制化开发	14
4. 配置远端开发环境	16
4.1 目的	16
4.2 环境说明.....	16
4.3 配置流程.....	16
4.3.1 安装 VsCode 软件	16
4.3.2 安装 SSH FS 插件	17
4.3.3 连接配置	17
4.3.4 SSH 连接.....	19
4.3.5 添加工作目录	19

表目录

表 1-1 EIC7x 软件栈匹配的开发板产品 1

表 2-1 EIC7x 软件栈中为用户提供的专属 deb 包 3

表 2-2 EIC7x 软件栈中为用户提供的工具清单（RISC-V 工具） 6

表 2-3 EIC7x 软件栈中为用户提供的工具清单（X86 PC 端工具） 6

表 3-1 EIC7x 软件栈用户文档 14

图目录

图 2-1 EIC7x 系列产品软件发行及用户使用方式图示..... 2

图 4-1 **VsCode 编辑器界面**..... 17

图 4-2 **安装 SSH FS 插件**..... 17

图 4-3 **新建 SSH FS 连接**..... 18

图 4-4 **SSH FS 配置文件的重要参数示例** 18

图 4-5 **VsCode 连接远端开发板** 19

图 4-6 **VsCode 配置远程开发板上的工作区** 19

1. 介绍 Introduction

本手册为 EIC7x 产品的软件开发用户如何使用该系列产品软件包的详细说明。

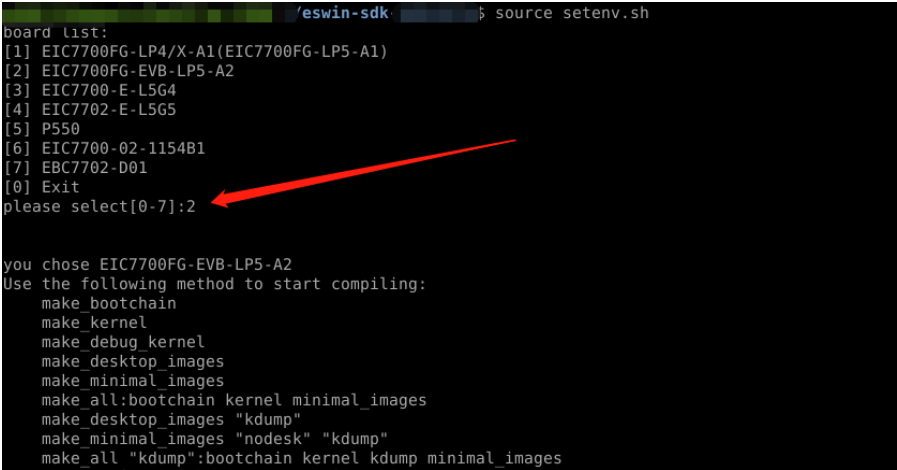
EIC7x 产品的开发者可以以本文档为基础，结合相应的开发板硬件、软件产品包、软件用户使用文档，进行定制化开发。客户要进行软件定制化开发，请参考本文第 3 章，尤其是第 3.3、3.4、3.5 节的内容进行。

软件产品的正确运行，需要配合正确的硬件配置，尤其是硬件连接方式、开发板中的拨码开关都需要参考用户使用的开发板对应的使用说明。

表 1-1 EIC7x 软件栈匹配的开发板产品

开发板型号 (新型号)	开发板型号 (老型号)	开发板名称	软件栈板型配置名称
P550	HF106	HiFive Premier P550 开发板	sifive_dvb_ddr5
EIC7700FG-LP4/X-A1	EIDS100A	EIC7700 LPDDR4X 评估板 V1.0	EIC7700-evb-a1
EIC7700FG-LP5-A1	EIDS200A	EIC7700 LPDDR5 评估板 V1.0	EIC7700-evb-a1
EIC7700FG-EVB-LP5-A2	EIDS200B	EIC7700 LPDDR5 评估板 V2.0	EIC7700-evb-a2
EIC7700-E-L5G4		EIC7700 LPDDR5 评估板 V3.0	EIC7700-evb-a3
EIC7702-E-L5G5	EIED100A	EIC7702 LPDDR5 评估板 V1.0	EIC7702-evb-a1
EIC7700-02-1154B1		EIC7700 LPDDR5 开发板 (314 pin SOM 板+底板)	EIC7700-d314
EBC7702-D01		EIC7702 LPDDR5 开发板 (560 pin SOM 板+底板)	EIC7702-d560

注1. 此处提供映射表，是为了保证客户更方便的使用开发板。待软件更加成熟后，板卡型号及配置文件将保持完全一致。



```
eswin-sdk $ source setenv.sh
board list:
[1] EIC7700FG-LP4/X-A1(EIC7700FG-LP5-A1)
[2] EIC7700FG-EVB-LP5-A2
[3] EIC7700-E-L5G4
[4] EIC7702-E-L5G5
[5] P550
[6] EIC7700-02-1154B1
[7] EBC7702-D01
[0] Exit
please select[0-7]:2

you chose EIC7700FG-EVB-LP5-A2
Use the following method to start compiling:
make_bootchain
make_kernel
make_debug_kernel
make_desktop_images
make_minimal_images
make_all:bootchain kernel minimal_images
make_desktop_images "kdump"
make_minimal_images "nodesk" "kdump"
make_all "kdump":bootchain kernel kdump minimal_images
```

2. EIC7x 软件栈完整生态整体介绍

奕斯伟提供的 EIC7x 芯片配套的软件栈，包括以下几方面内容：

/softwares	-----包含构建 minimal 和 desktop 系统的脚本文件
/pc_tools	-----X86 平台上位机的工具，参考第 2.2.2 节。。
/nn_tools	-----为用户提供的 AI 工具链 Docker 镜像，参考第 2.2.3 节。
/docs	-----软件栈文档，参考表 3-1。

我们为用户提供的 Debian 发行版，遵照 Linux 系统要求，以及标准 Debian 发行版的规范。具体使用方式方法，请用户查阅 Linux 系统和标准 Debian 发行版的用户使用文档。本文仅为用户提供软件栈的使用、二次开发必须的使用说明。二次开发涉及本产品的软件栈相关的内容，请用户参考本文第 3 章罗列的文档进行。

2.1 Debian 发行版的组织形式

EIC7x 芯片配套的软件栈，包括 Debian 发行版，以及 pc 端运行的各种工具（包括运行在 X86 平台上的 windows 平台的工具和 linux 平台的工具）。其中，直接提供给用户的 Debian 发行版，是一个 minimal 的可以通过网络直接安装的发行版。用户也可以通过软件包中默认的脚本，直接从网络获取 ESWIN 按照标准完全版本形式，为用户提供的包含 GUI、浏览器等多种桌面应用的软件包。

用户可以在 minimal Debian 发行版基础上，根据自己的使用需要，通过脚本从 PLCT 维护的 Debian 软件仓库、奕斯伟提供的专属应用仓库中获取所需的软件包；也可以在最小 Debian 发行版的基础上，按照用户定制的方式进行开发。

如果客户编译内容存在外部依赖，基于现有Debian发行版的操作流程

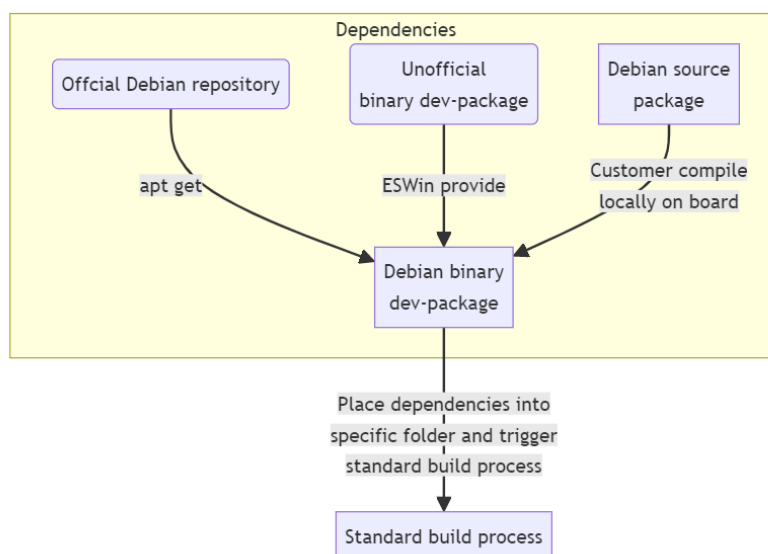


图 2-1 EIC7x 系列产品软件发行及用户使用方式图示

2025 年的 06/30 版本为用户提供了 minimal 发行版和桌面版两个发行镜像。

注2. minimal 与桌面版本的系统默认账号/密码均是 eswin/eswin

另外，查看 boot.ext4 版本和 root.ext4 版本的方法示例如下所示：

```
eswin@rockos-eswin:~$ uname -a
Linux rockos-eswin 6.6.18-eic7x-2024.11 #2024.11.21.02.05+ SMP Thu Nov 21 02:16:13
UTC 2024 riscv64 GNU/Linux

eswin@rockos-eswin:~$ cat /etc/*release
EIDS200B:EIC7X-2024.11
```

2.2 EIC7x 软件栈为用户提供的专属工具

2.2.1 Eswin 提供的专属 deb 包

ESWIN EIC7x 软件栈为用户提供了表 2-1 罗列的专属 deb 包。要正确使用这些为用户提供的软件包，请用户按照本文第 3 章的顺序，并在第 3.5 节中详细了解每个包的具体使用注意事项（如未提供，则不需要用户特别注意）。

表 2-1 EIC7x 软件栈中为用户提供的专属 deb 包

专属 deb 包名称	用途说明	注意事项
es-sdk-log_1.11.0_riscv64.deb	ESSDK 日志模块，提供给各个模块的日志输出库	
es-sdk-memory_1.11.0_riscv64.deb	通用内存和 vb 内存的分配管理接口	● 要在某一个内存区域（比如：mmz_nid_0_part_0）分配内存，请确保内存分配前，相关的内存区域已在 dts 中被描述。
es-sdk-memcp_1.11.0_riscv64.deb	EsSDK Memcp 模块	
es-sdk-cipher_1.11.0_riscv64.deb	ESSDK Cipher 模块	
es-sdk-numa_1.11.0_riscv64.deb	ESWIN NUMA 模块	
es-sdk-common_1.11.0_riscv64.deb	ESSDK 的公共头文件	
es-sdk-video-utils_1.11.0_riscv64.deb	ESSDK 中 Video 模块内部使用的公共工具库	

es-sdk-sys_1.11.0_riscv64.deb	ESSDK SYS 模块	● 安装后，es_media_srv 会自动后台运行起来，不需要再次启动。重启之后，es_media_srv 不会自动运行； ● 如果应用程序调用了 ES_SYS_Init 接口，程序运行的时候会把 es_media_srv 拉起来，否则也不需要 es_media_srv 的功能。
es-sdk-video_1.11.0_riscv64.deb	ESSDK Video 模块，包含 vdec、venc、vps、vo	
es-hae_1.11.0_riscv64.deb	ESWIN 图像处理硬件加速引擎	
es-video-common_1.11.0_riscv64.deb	ESWIN Video 公共库	
es-mpp_1.11.0_riscv64.deb	ESWIN 多媒体处理平台	
es-sdk-isp_1.11.0_riscv64.deb	ESWIN 图像处理 ISP 模块	
es-sdk-npu_1.11.0_riscv64.deb	ESWIN NPU 智能推理模块，包含 NPU 基础设施库和文件具体包括：NPU Runtime 库和 E31 firmware	
es-sdk-dsp_1.11.0_riscv64.deb	ESWIN DSP 智能推理模块，包含 NPU 基础设施库和文件具体包括：DSP Runtime 库和 DSP firmware	
es-sdk-audio_1.11.0_riscv64.deb	ESSDK Audio 模块，包含音频处理各种 Component，AI，AO，AENC，ADEC 等模块	
es-sdk-audio-codec_1.11.0_riscv64.deb	ESSDK Audio 编解码库，包含 AAC-LC/HEv1/v2，AMR-Nb/Wb，MP3，G7XX、MP2L2 解码和 AAC-LC 编码器	
es-sdk-ak_1.11.0_riscv64.deb	AcceleratorKit 是用于调用未编译入 ONNX 模型的算子的 API 库，如一些较为复杂的前处理、后处理算子等	
ffmpeg.deb	ffmpeg7.0 的安装包，里面集成了 eswin 实现的硬件编解码插件	

es-sdk-sample-audio_1.11.0_riscv64.deb	ESSDK Audio SDK 接口调用 Sample	
es-sdk-sample-video_1.11.0_riscv64.deb	ESSDK video sample	
es-sdk-sample-memory_1.11.0_riscv64.deb	es-sdk-memory 包的使用示例	- sample 程序中使用了 spram 区域，如果此区域被 npu 程序占用，则相关 case 会执行失败； - IOVA 相关的接口依赖对应模块的驱动程序，请确保调用前对应的驱动已经被加载。
es-sdk-sample-dsp_1.11.0_riscv64.deb	es-sdk-dsp 包的使用示例	
es-sdk-sample-cipher_1.11.0_riscv64.deb	es-sdk-cipher 包的使用示例	
es-sdk-sample-memcp_1.11.0_riscv64.deb	es-sdk-memcp 包的使用示例	
es-fand_1.11.0_riscv64.deb	风扇控制模块	
escl-server_1.11.0_riscv64.deb	提供 ESWIN 多媒体处理平台 mpp 调用 vendor 接口的功能	
es-hw-watcher_1.11.0_riscv64.deb	动态获取设备硬件信息（温度，电压，占用率，带宽等具体参考 help 或 ReadMe）以及动态展示，支持设备：NPU、VDEC、VENC	
es-lightdm-numactl_1.11.0_riscv64.deb	桌面显示优化，优化显示模块性能	优化只针对多 numa 平台，请勿安装到单 numa 平台
es-numa-manager_1.11.0_riscv64.deb	桌面系统优化，优化前台窗口性能	
es-performance_1.11.0_riscv64.deb	接口和工具性能优化	
es-boottime-optimizer_1.11_riscv64.deb	ESWIN 加快开机速度	针对 ubuntu

2.2.2 X86 平台上使用的工具清单

ESWIN EIC7x 软件栈，在 X86 平台上为用户提供了如下工具，这些工具都是使用在 linux 环境下。

表 2-2 EIC7x 软件栈中为用户提供的工具清单（RISC-V 工具）

工具名称	工具用途	备注
Nsign 工具	Nsign 是一款能保障要加载镜像安全性和完整性的镜像制作签名工具。包含以下主要功能：摘要计算、对称加密、非对称加密、多级镜像签名打包以及非对称加密密钥的生成。	具体使用方法请参考文档《镜像制作和 EsNsign 用户手册》。 工具源码 github 路径： https://github.com/eswincomputing/Esbd-77serial-nsign

2.2.3 为用户单独提供的 Docker 镜像

ESWIN EIC7x 软件栈，给用户三个 docker 镜像。这三个镜像因为容量巨大，单独给用户。

表 2-3 EIC7x 软件栈中为用户提供的工具清单（X86 PC 端工具）

软件工具包名称	工具用途	备注
esquant_docker.tar	EsQuant Docker 镜像	具体使用方法请参考文档“ENNP 用户手册”
esaac_essimulator_docker.tar	EsAAC、EsSimulator 工具的 Docker 镜像	具体使用方法请参考文档“ENNP 用户手册”
es_debian_compile_docker.tar	用于编译 debian 环境的 Docker 镜像	具体使用方法请参考本文档下一章
esquant-1.0-py3-none-any.whl	EsQuant 工具 python 安装包	具体使用方法请参考文档“ENNP 用户手册”

注3. ESWIN 提供 EIC7x 系列产品的标准软件发布镜像（通过公开渠道发布）。因 Docker 镜像太大，暂不直接公开给个人用户公开发布。用户如果需要，请联系销售渠道。

3. EIC7x 软件栈使用

3.1 环境搭建

准备一个 ubuntu22.04 机器或者 ubuntu20.04，网络配置上需要能访问 github.com。

若使用 ubuntu22.04 系统，需要安装以下软件：

```
sudo apt install -y gdisk dosfstools build-essential libncurses-dev gawk flex bison openssl  
libssl-dev tree dkms libelf-dev libudev-dev libpci-dev libliberty-dev autoconf device-tree-  
compiler xz-utils devscripts ccache debhelper wget curl pahole libconfuse-dev mtools  
fastboot rsync  
  
sudo apt install -y debootstrap devscripts qemu-user-static qemu-utils binfmt-support  
mmdebstrap  
  
wget https://github.com/riscv-collab/riscv-gnu-toolchain/releases/download/  
d/2024.04.12/riscv64-glibc-ubuntu-22.04-gcc-nightly-2024.04.12-nightly.tar.gz  
sudo tar -xvf riscv64-glibc-ubuntu-22.04-gcc-nightly-2024.04.12-nightly.tar.gz -C /opt
```

若使用 ubuntu20.04，则可以直接使用 docker 进行编译，将 es_debian_compile_docker.tar 上传到 ubuntu20.04 的环境上，加载好 docker 镜像。

```
sudo docker load -i es_debian_compile_docker.tar  
sudo docker pull multiarch/qemu-user-static  
sudo docker run --rm --privileged multiarch/qemu-user-static --reset -p yes
```

3.2 烧写

用户完成基础环境搭建之后，就可以参考第 3.3 节的内容进行定制化开发。当完成定制化开发的编译后，就需要用户将编译的结果烧写到开发板中。

用户在开发过程中涉及反复执行“开发→编译→烧写”的过程，因此将“烧写”过程提前。用户烧写板卡的方式，因选购的产品类型不同而不同，包括：板卡跟上位机的连接方式、不同外设的连接方式和端口、板卡中拨码开关的设置或跳线配置方式等。为了确保烧写过程不出错，用户需要根据购买产品的型号，按不同板型开发板产品中为用户提供的“开发板入门指南”文档中的“下载烧录”章节进行操作。

注4. 用户在进行烧写的时候，板卡跟 X86 平台的上位机的连接配置、以及板卡上的拨码开关、跳线等的连接方式务必按照板卡对应的使用说明进行。不正确的连接方式可能造成板卡的不可预知的损坏，影响您的使用。

3.3 系统编译

将发布包中的 eswin-sdk-*.tar.gz 包上传到 ubuntu22 或者 ubuntu20 上并解压：

```
tar -xf eswin-sdk-*.tar.gz -C $HOME
```

解压后进入到对应目录，可看到 source 目录和 setenv.sh 脚本，source 目录在编译过程中会存放从 github 上下载的 linux、opensbi、uboot 的源码，编译过程中会自动下载，此过程中可能因网络问题导致下载失败，可通过如下命令先将源码提前下载好。

注：请将如下命令中 EIC7X-2025.x 中的“x”替换为当前发布的版本，例如 0830 版本替换为“08”，1030 版本替换为“10”，1130 版本替换为“11”，以此类推。

```
git clone --depth=1 -b EIC7X-2025.12https://github.com/eswincomputing/u-boot.git
source/uboot-eswin
git clone --depth=1 -b EIC7X-2025.12https://github.com/eswincomputing/opensbi.git
source/opensbi-eswin
git clone --depth=1 -b EIC7X-2025.12https://github.com/eswincomputing/linux-stable.git
source/linux-eswin
```

Ubuntu20.04 的环境，在解压后的目录下执行如下命令启动 docker 容器

```
sudo docker run --rm --privileged -it -v "$(pwd)":/home/workspace -u root -w
"/home/workspace" es_debian
```

进入解压后的目录执行如下命令：

```
source setenv.sh
```

根据版型选择：

```
root@4c095b1c0404:/home/workspace# source setenv.sh
board list:
[1] EIC7700FG-LP4/X-A1(EIC7700FG-LP5-A1)
[2] EIC7700FG-EVB-LP5-A2
[3] EIC7700-E-L5G4
[4] EIC7702-E-L5G5
[5] P550
[6] EIC7700-02-1154B1
[0] Exit
please select[0-6]:5
you chose P550
Use the following method to start compiling:
make_bootchain
make_kernel
make_debug_kernel
make_desktop_images
make_minimal_images
make_all:bootchain kernel minimal_images
```

- ◆ make_bootchain：编译 bootloader，生成 bootloader*.bin 和 recover*.bin。recover*.bin 用户参考《开发板镜像安装与升级手册》第 2.2 节
- ◆ make_kernel：编译 kernel，生成 deb 包，编译 root.ext4 和 boot.ext4 时会用到生成的 deb
- ◆ make_minimal_images：生成一个精简版的 root.ext4(1000M)和 boot.ext4(100M)
- ◆ make_desktop_images：生成一个包含 desktop 的 root.ext4(7G)和 boot.ext4(500M)
- ◆ make_all：编译 bootloader、和精简版的系统

例如需要快速编译一个精简版的系统：

```
make_all
```

编译过程最后，制作 images 这个流程需要 sudo 权限，需要输入一次 sudo 的密码，使用 docker 镜像不会提示输入密码：

```
start compile debian mkimg
[sudo] 的密码：
```

编译完成后在当前目录可看到编译生成的 bootloader.bin root.ext4 boot.ext4：

```

drwxr-xr-x 2 root root      4096 Apr 21 05:44 ./
drwxr-xr-x 8 root root      4096 Apr 21 03:13 ../
-rw-r--r-- 1 root root 104857600 Apr 21 02:59 boot-P550-20250421-025126.ext4
-rw-r--r-- 1 root root 4306584 Apr 21 02:49 bootloader_P550.bin
-rwxr-xr-x 1 root root 4030200 Apr 21 02:49 fw_payload.bin*
-rw-r--r-- 1 root root 8603052 Apr 21 02:51 linux-headers-6.6.18-eic7x-2025.03_6.6.18-2025.04.21.02.49+_riscv64.deb
-rw-r--r-- 1 root root 22371792 Apr 21 02:51 linux-image-6.6.18-eic7x-2025.03_6.6.18-2025.04.21.02.49+_riscv64.deb
-rw-r--r-- 1 root root 1336454 Apr 21 02:51 linux-libc-dev_6.6.18-2025.04.21.02.49+_riscv64.deb
-rw-r--r-- 1 root root 5631544 Apr 21 02:49 recovery_bootloader_P550.bin
-rw-r--r-- 1 root root 1048576000 Apr 21 02:59 root-P550-20250421-025126.ext4
-rw-r--r-- 1 root root 1933042 Apr 21 02:48 u-boot.bin
-rw-r--r-- 1 root root 42418 Apr 21 02:48 u-boot.dtb

```

编译带桌面的系统版本，已编译好 kernel，可省略 “make_kernel”：

```

make_kernel
make_desktop_images

```

3.4 Hello world 示例程序

通过环境搭建章节，可知交叉编译工具链存放于/opt/riscv 目录下，当在执行完

“source setenv.sh” 后，系统上已经添加如下环境变量：

```

export PATH="/opt/riscv/bin:$PATH"
export ARCH=riscv
export CROSS_COMPILE=riscv64-unknown-linux-gnu-

```

本节将指导用户通过使用交叉编译工具链编译一个简单的 hello world 程序、以及如何在工具链中新增三方包。

3.4.1 交叉编译

新建目录 hello，在该目录下新建文件 hello.c 和 Makefile。

hello.c

```

#include <yaml.h>
int main() {
    printf("Hello, world!\n");
    return 0;
}

```

Makefile

```

CC = $(CROSS_COMPILE)gcc
CFLAGS = -Wall
TARGET = hello
SRC = hello.c
OBJ = $(SRC:.c=.o)

all: $(TARGET)

$(TARGET): $(OBJ)
    $(CC) $(CFLAGS) -o $@ $^

%.o: %.c
    $(CC) $(CFLAGS) -c -o $@ $<

clean:
    rm -f $(OBJ) $(TARGET)

```

执行交叉编译

```
make
```

完成交叉编译后会生成可执行程序 hello，将 hello 程序拷贝到烧录好的系统上测试：

```
root@~$ ./hello  
Hello, world!
```

3.4.2 构建 deb 包

当编译好的程序包含 lib、bin、include 等多种文件，并且需要安装到系统的不同目录时，可将该程序构建成一个 deb 包，以上述 hello word 程序为例，假如编译好的的 hello 程序，有可执行程序 hello、头文件 hello.h、lib 文件 libhello.so，要求将 hello 安装到/usr/bin，libhello.so 安装到/usr/lib，hello.h 安装到/usr/include/，可通过如下流程构建 deb 包：

1. 完成交叉编译
2. 编辑 deb 包构建脚本

```
mkdir -p debian/work/DEBIAN  
cd debian/work  
mkdir -p usr/lib  
mkdir -p usr/bin  
mkdir -p usr/include  
cp ../../hello usr/bin  
cp ../../libhello.so usr/lib  
cp ../../hello.h usr/include  
echo Package: hello >>DEBIAN/control  
echo Version: 1.0 >>DEBIAN/control  
echo Section: utils >>DEBIAN/control  
echo Priority: optional >>DEBIAN/control  
echo Architecture: riscv64 >>DEBIAN/control  
echo Depends: >>DEBIAN/control  
echo Installed-Size: 10240 >>DEBIAN/control  
echo Maintainer: robot@eswincomputing.com >>DEBIAN/control  
echo Description: essdk hello >>DEBIAN/control  
dpkg -b ../../hello-V1.0.deb
```

新建 hello_deb.sh，将以上内容保存到 hello_deb.sh 中，并给予可执行权限。

```
hello  
├── hello  
├── hello.c  
├── hello_deb.sh  
├── hello.h  
├── hello.o  
├── libhello.so  
└── Makefile  
  
0 directories, 7 files
```

3. 生成 deb 包

```
./hello_deb.sh
```

完成后当前目录下可看到生成的 hello-V1.0.deb

4. 安装 deb 包

将 hello-V1.0.deb 上传到目标系统中执行安装：

```
sudo dpkg -i --force-all hello-V1.0.deb
```

5. 检查 deb 部署是否符合预期

```
hello ~$ ls /usr/bin/ | grep hello
hello
hello ~$ ls /usr/lib/ | grep hello
libhello.so
hello ~$ ls /usr/include/ | grep hello
hello.h
hello ~$ hello
Hello, world!
hello ~$
```

3.4.3 交叉编译工具链扩展

交叉编译一些程序时，可能出现找不到一些开源包，因此需要在交叉编译工具链中增加这些开源包，以 hello 程序为例，新增对 libyaml 的引用，执行交叉编译时会出现如下错误：

```
~/hello$ make
riscv64-unknown-linux-gnu-gcc -Wall -c -o hello.o hello.c
hello.c:2:10: fatal error: yaml.h: No such file or directory
  2 | #include <yaml.h>
    |          ^~~~~~
compilation terminated.
make: *** [Makefile:13: hello.o] 错误 1
```

可通过如下步骤在交叉编译工具链中新增 libyaml：

1. 备份交叉编译工具链的 sysroot

```
sudo mv /opt/riscv/sysroot/ /opt/riscv/sysroot_bak
```

2. 挂载编译好的 root.ext4

```
mkdir rootfs
sudo mount root.ext4 rootfs/
```

创建的一个 rootfs 目录，将 root.ext4 挂载到 rootfs 目录下。

minimal 的 root.ext4 空间较小，apt install 过程中可能会提示空间问题，可此步骤前，在 ubuntu22.04 上通过 resize2fs（sudo apt-get install e2fsprogs）命令调整，例如：

```
sudo resize2fs root.ext4 7G
```

3. 指定新的 sysroot 目录

制定新的 sysroot 目录，将创建的 rootfs 目录软连接到/opt/riscv/sysroot

```
sudo ln -s /x/x/x/x/rootfs/ /opt/riscv/sysroot
```

4. chroot 安装 libyaml


```
sudo chroot rootfs/  
apt install gcc  
apt install build-essential  
cd /usr/include  
ln -s riscv64-linux-gnu/bits bits  
ln -s riscv64-linux-gnu/sys sys  
ln -s riscv64-linux-gnu/gnu gnu  
ln -s riscv64-linux-gnu/asm asm  
cd /usr/lib  
cp riscv64-linux-gnu/* ./ -rf
```

chroot 后会进入到一个虚拟的系统中，在这个系统中可通过 apt 查找、安装需要的包，如需要查找支持开发的 libyaml。

```
apt search libyaml
```

```
root@riscv64-unknown-linux-gnu:/# apt search libyaml  
erlang-pl-yaml/sid 1.0.36-2 riscv64  
  erlang wrapper for libyaml C library  
  
libghc-libyaml-dev/sid,now 0.1.2-3+b3 riscv64 [installed]  
  low-level, streaming YAML interface.  
  
libghc-libyaml-doc/sid 0.1.2-3 all  
  low-level, streaming YAML interface.; documentation  
  
libghc-libyaml-prof/sid 0.1.2-3+b3 riscv64  
  low-level, streaming YAML interface.; profiling libraries  
  
libghc-yaml-dev/sid 0.11.11.2-1+b3 riscv64  
  interface to LibYAML  
  
libghc-yaml-doc/sid 0.11.11.2-1 all  
  interface to LibYAML; documentation  
  
libghc-yaml-prof/sid 0.11.11.2-1+b3 riscv64  
  interface to LibYAML; profiling libraries  
  
librust-unsafe-libyaml-dev/sid 0.2.11-1 riscv64  
  Libyaml transpiled to rust by c2rust - Rust source code
```

找到对应的开发包，带“-dev”后缀的安装后会有头文件：

```
apt install libghc-yaml-dev
```

安装完成后交叉编译工具链新增了 libyaml 的 lib、include 等，回到交叉编译环境上，再次执行编译：

```
/hello$ make  
riscv64-unknown-linux-gnu-gcc -Wall -o hello hello.o
```

3.5 EIC7x 软件栈中 ESWIN 提供的专属 Deb 包的使用方法及注意事项

3.5.1 安装软件 Deb 包

用户通过 PLCT 的 Debian 包源，在线安装 eswin 提供如表 2-1 中所列的 debian 包，具体安装方式如下：

```
sudo apt update
sudo apt install es-sdk-log
sudo apt install es-sdk-memory
sudo apt install es-sdk-memcp
sudo apt install es-sdk-cipher
sudo apt install es-sdk-numa
sudo apt install es-sdk-common
sudo apt install es-sdk-video-utils
sudo apt install es-sdk-sys
sudo apt install es-sdk-video
sudo apt install es-hae
sudo apt install es-video-common
sudo apt install es-sdk-isp
sudo apt install es-mpp
sudo apt install es-sdk-npu
sudo apt install es-sdk-dsp
sudo apt install es-sdk-audio
sudo apt install es-sdk-audio-codec
sudo apt install es-sdk-ak
sudo apt install ffmpeg
sudo apt install es-sdk-sample-dsp
sudo apt install es-sdk-sample-audio
sudo apt install es-sdk-sample-video
sudo apt install es-sdk-sample-memory
sudo apt install es-sdk-sample-cipher
sudo apt install es-sdk-sample-memcp
sudo apt install es-fand
sudo apt install es-boottime-optimizer
```

如果安装过程中报空间不足，则需要调整文件系统分区的大小，执行以下命令：

```
sudo resize2fs /dev/mmcblk0p3
```

注：若特定发布的版本中提供了离线 deb 安装包，如表 2-1 中所示，则用户可以选择离线安装，具体在线安装还是离线安装，客户可根据诗句情况选择。离线安装方式如下所示：

```
sudo apt install *.deb
```

3.6 定制化开发

EIC7x 系列芯片的软件栈，为用户提供了从底层到应用的完整定制开发的环境和资源。用户根据产品的类型、场景，基于软件栈提供的这些资源，进行定制化开发。

客户如果需要面向上层应用的定制化开发，可以直接使用为用户提供的 ESSDK sample 的资源做二次开发。客户如果需要从底层就开始做定制化开发，则可以基于我们发行的软件，从官方公布的渠道，获得对应的源码，结合用户自己业务需求进行二次开发。

定制化开发过程和方法，参考本文的第 3.4 节“Hello world 示例程序”进行具体软件包的开发编译。完整镜像的定制化开发，则结合本章前述章节，并结合开发板硬件使用指南完成镜像的烧写。

EIC7x 系列产品的软件栈，包含系统、音视频、智能等几个方面，几个方面分别包含的文档如下表 3-1 所示，用户在进行定制化开发过程中，直接参考对应文档的内容即可。

表 3-1 EIC7x 软件栈用户文档

分类	文档名称	备注
系统	Cipher 用户手册	Cipher 是 EIC7x SOC 平台提供的安全算法模块，其提供了包括 AES、DES/3DES 和 SM4 等对称加解密算法，RSA、ECSDA 不对称加解密算法，随机数生成，以及支持 HASH、HMAC、MAC 等摘要算法，主要用于对音视频码流进行加解密保护，用户合法性认证等场景。
系统	Memory 用户手册	Memory 模块主要向媒体业务提供大块物理内存管理功能，负责内存缓存池、内存块的分配和回收，让物理内存资源在各个媒体处理模块中合理使用。
系统	Memory_Copy 用户手册	MemCP（Memory Copy）模块提供了面向系统和应用程序的统一内存复制与命令队列管理接口。通过该模块，用户可以实现高效的异步数据传输和任务调度，支持复杂内存操作在多任务并发环境中的高效执行。 MemCP 模块内核实现了命令队列（CMDQ）与内存复制（Memcpy）功能，提供了一致性强、易于使用的用户层接口。
系统	SYS 开发手册	SYS（system control）模块主要为 ESSDK 提供公共基础服务，主要包括系统管理、版本管理、内存管理、日志管理、时间管理、绑定管理功能。

系统	Bootloader 移植应用开发指南	EIC7x Bootloader 主要由 U-Boot 和 opensbi 组成，用户在开发移植过程中只需要关注 U-Boot 即可，该手册主要描述 U-Boot 下载、编译、移植、opensbi 下载编译以及常用的 U-Boot 命令。
系统	开发板镜像安装与升级手册	该手册介绍了如何烧写、升级系统镜像的方法和操作流程。
系统	镜像制作和 EsNsign 用户手册	该手册介绍了如何使用 Nsign 工具，打包系统镜像的操作流程。
系统	软件业务场景功耗调节指南	该文档介绍了在软件层面如何调节 EIC7x 系列开发板功耗的方法。
音频	音频用户手册	Audio 模块包括音频输入、音频输出、音频编码、音频解码四个子模块。音频输入和输出模块通过对 EIC7x 芯片音频接口的控制实现音频输入输出功能。音频编码和解码模块提供对 G711、G722、G726、AAC、MP3、AMR 等格式的音频编解码功能，并支持录制和播放 LPCM 格式的原始音频文件。
音频	音频开发手册	Audio 模块包括音频输入、音频输出、音频编码、音频解码四个子模块。音频输入和输出模块通过对 EIC7x 芯片音频接口的控制实现音频输入输出功能。音频编码和解码模块提供对 G711、G722、G726、AAC、MP3、AMR 等格式的音频编解码功能，并支持录制和播放 LPCM 格式的原始音频文件。
视频	VPS 开发手册	VPS（Video process system）是视频图像处理系统，以分时复用的方式提供图像处理功能。包括鱼眼矫正、畸形矫正、裁剪、缩放、颜色空间转换、固定角度旋转、MIRROR、FLIP、画线、纯色填充、归一化、帧率控制、OVERLAY、幅型比等功能。
视频	VENC 开发手册	VENC 模块，即视频编码模块。
视频	VDEC 开发手册	VDEC 模块负责对 H.264/265、JPEG 格式的数字压缩视频进行解压缩处理，从而得到 YUV/RGB 格式的图像流。该模块支持多路解码。
视频	VO 开发手册	VO 模块主动从内存响应位置读取视频数据，并通过显示设备输出视频。
音频\视频	FFMPEG 用户手册	FFMPEG 插件主要包含编解码的插件使用方法，以及 Mpv 和 obs 简单使用流程。
智能	ENNP 用户手册	ENNP（ESWIN NetWork Processing）平台是 EIC7x SOC 芯片智能计算异构加速平台，包含了对 NPU、DSP、HAE 以及 GPU 的开发工具及接口。

4. 配置远端开发环境

4.1 目的

配置远程开发环境的主要目的，是为了方便开发者在本地计算机上利用远程开发板（EIC7x 系列）的计算资源和环境，进行应用开发、代码编译、工程部署等操作。

4.2 环境说明

选择“VsCode + 插件”的方式，理论上可以安装 VsCode 软件的开发机，都可以作为本地环境的开发主机。远端环境，即 EIC7x 系列开发板，包括表 1-1 中的所有型号。本配置方案以如下开发环境为例，介绍整个远端开发环境的搭建流程。

本地环境：ubuntu20.04 系统开发主机

远端环境：EIC7x 系列开发板

4.3 配置流程

4.3.1 安装 VsCode 软件

(1) 首先 apt 命令更新软件包索引并安装导入微软 GPG 密钥的依赖软件；

```
sudo apt update
```

(2) 安装工具包；

```
sudo apt install software-properties-common apt-transport-https curl
```

(3) 当导入 GPG 后，运行 curl 命令导入 Microsoft GPG 密钥；

```
curl -sSL https://packages.microsoft.com/keys/microsoft.asc | sudo apt-key add -
```

(4) 将 vscode 软件源添加到 Ubuntu 22.04 软件源列表；

```
sudo add-apt-repository "deb [arch=amd64]  
https://packages.microsoft.com/repos/vscode stable main"
```

(5) 安装 vscode

```
sudo apt update  
sudo apt install code
```

(6) 启动 Vscode

在终端中输入 code，回车，即可启动 VsCode 编辑器页面，如下图 4-1 所示。

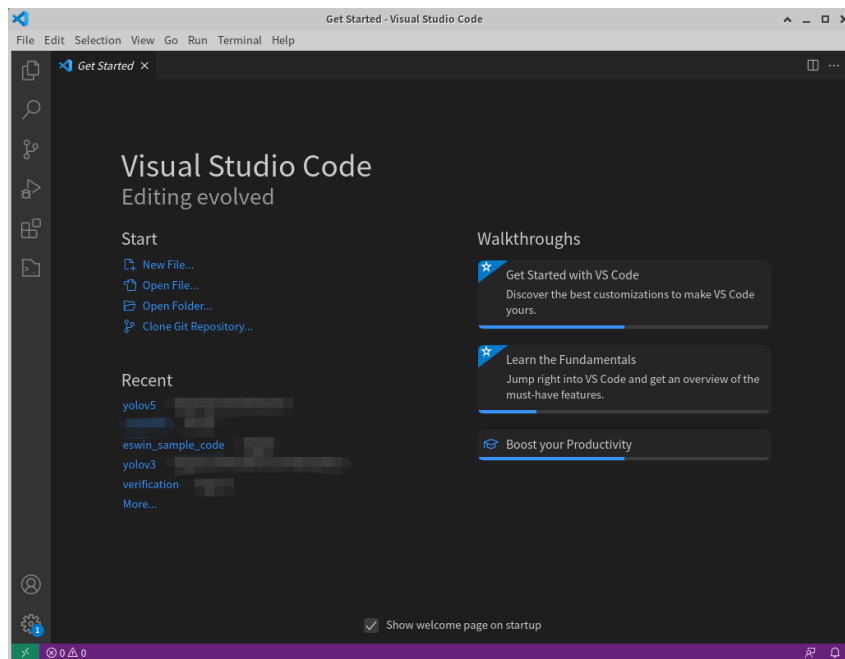


图 4-1 VsCode 编辑器界面

4.3.2 安装 SSH FS 插件

按如下图 4-2 示，安装 SSH FS 插件：

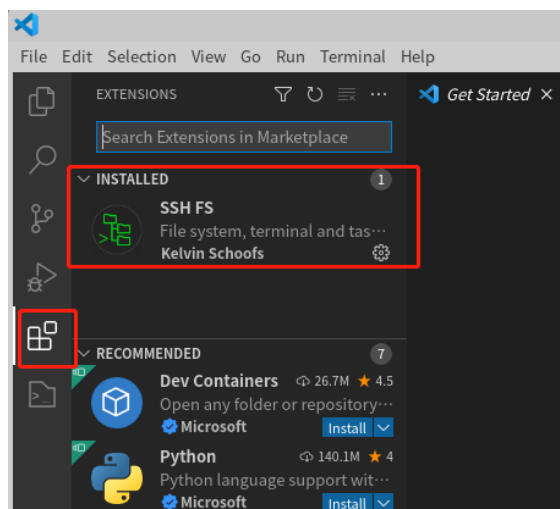


图 4-2 安装 SSH FS 插件

4.3.3 连接配置

在 CONFIGURATIONS 下点击 Create a SSH FS Configuration 创建一个新的连接，设定名字后保存。如下图 4-3 所示：

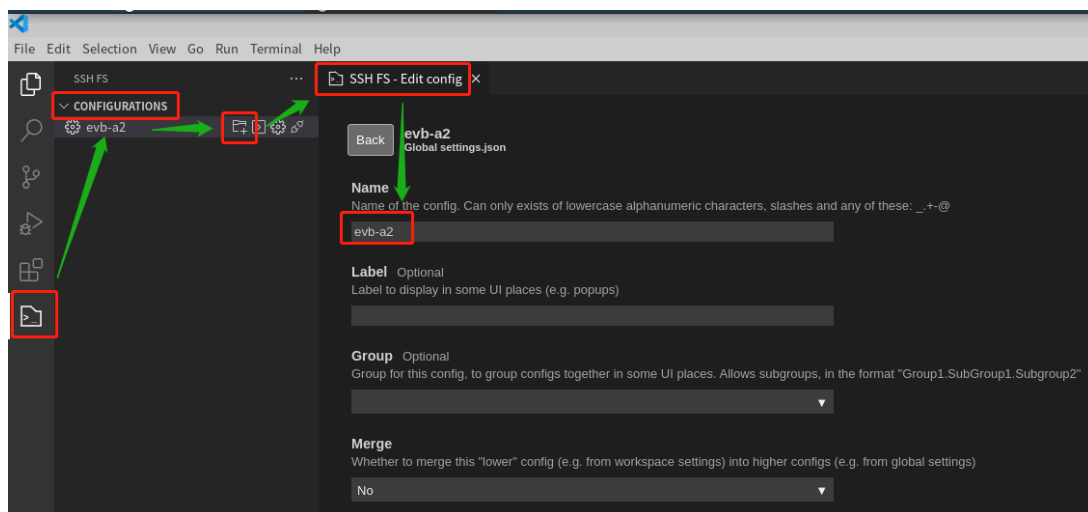


图 4-3 新建 SSH FS 连接

在配置文件中，必须配置的重要参数如下图 4-4 所示：

Host Optional
Hostname or IP address of the server. Supports environment variables, e.g. \$HOST
10.10.194.179

Port Optional
Port number of the server. Supports environment variables, e.g. \$PORT
22

Root Optional
Path on the remote server that should be opened by default when creating a terminal or using the 'Add as Workspace folder' command/button. Defaults to '/'
/

Agent Optional
Path to ssh-agent's UNIX socket for ssh-agent-based user authentication. Supports 'pageant' for PuTTY's Pageant, and environment variables, e.g. \$SSH_AUTH_SOCK
pageant

Username Optional
Username for authentication. Supports environment variables, e.g. \$USERNAME
eswin

Password Optional
Password for password-based user authentication. Supports env variables. This gets saved in plaintext! Using prompts or private keys is recommended!
eswin

图 4-4 SSH FS 配置文件的重要参数示例

如上图 4 所示，其中：

Host：填写 EIC7x 开发板的 ip；

Root：填写远程登录到开发板之后，在终端显示的工作区路径；

Username：开发板的登录用户名称；

Password：开发板登录用户对应的密码。

4.3.4 SSH 连接

连接配置创建之后，在 VSCode 界面中通过如下图 4-5 所示，鼠标点击 “>” 控件之后，在 VSCode 终端上直接 SSH 登录远程开发板环境中，然后即可在此终端下进行远程开发了。

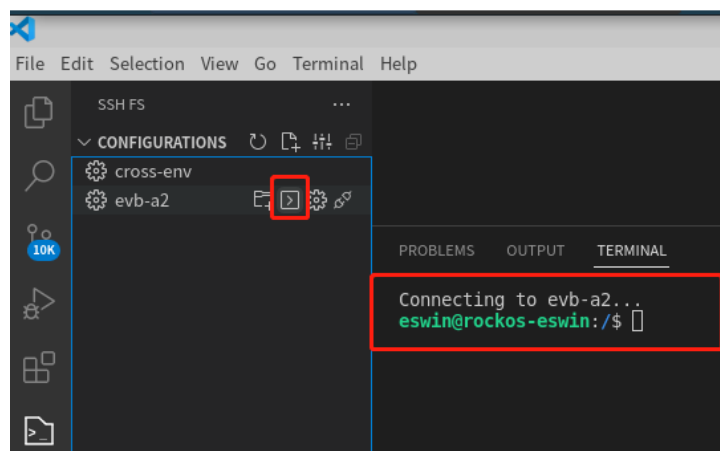


图 4-5 VSCode 连接远端开发板

4.3.5 添加工作目录

在 VSCode 软件界面同样可以配置 “workspace” 工作区，开发者就可以在本地编写、修改、编译远程开发板上的代码工程。添加工作目录流程如下图 4-6 所示：

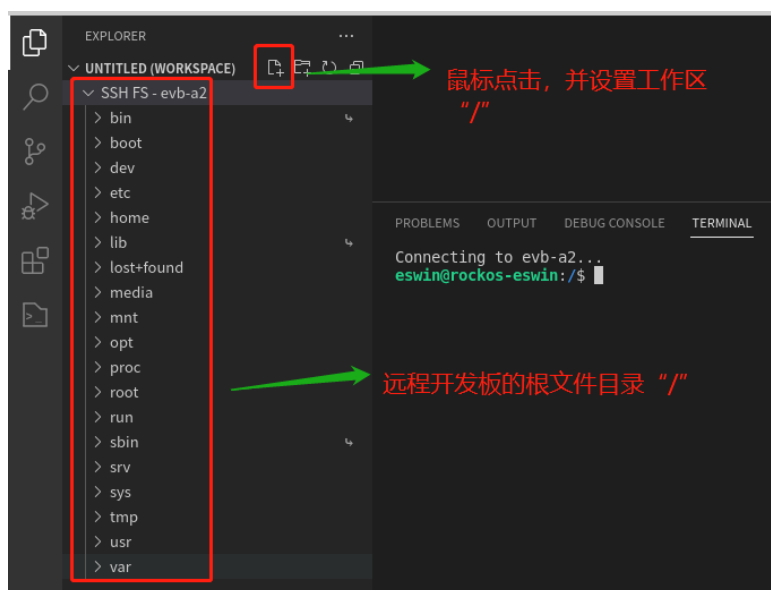


图 4-6 VSCode 配置远程开发板上的工作区